

DynaHash: An efficient blocking structure for streaming record linkage

Dimitrios Karapiperis^a, Christos Tjortjis^a, Vassilios S. Verykios^b

^a International Hellenic University, Greece

^b Hellenic Open University, Greece

ARTICLE INFO

Keywords:

Record linkage
Locality-sensitive hashing
Multi-Probe matching

ABSTRACT

Record linkage holds a crucial position in data management and analysis by identifying and merging records from disparate data sets that pertain to the same real-world entity. As data volumes grow, the intricacies of record linkage amplify, presenting challenges, such as potential redundancies and computational complexities. This paper introduces DynaHash, a novel randomized record linkage mechanism that utilizes (a) the MinHash technique to generate compact representations of blocking keys and (b) Hamming Locality-Sensitive Hashing (LSH) to construct the blocking structure from these vectors. By employing these methods, DynaHash offers theoretical guarantees of accuracy and achieves sublinear runtime complexities, with appropriate parameter tuning. It comprises two key components: a persistent storage system for permanently storing the blocking structure to ensure complete results, and an in-memory component for generating very fast partial results by summarizing the persisted blocking structure. Additionally, DynaHash leverages Multi-Probe matching to scan multiple neighboring blocks, in terms of their Hamming distances, in order to find matches. Our theoretical work derives a decrease factor in the space requirements, which depends on the Hamming threshold, compared with the baseline LSH. Our experimental evaluation against three state-of-the-art methods on six real-world data sets demonstrates DynaHash's exceptional recall rates and query times, which are at least 2× faster than its competitors and do not depend on the size of the underlying data sets.

1. Introduction

Record linkage is a pivotal technique utilized across various data management and analysis endeavors. Essentially, it involves the identification of matching records from multiple data sets that correspond to the same real-world entity. However, the large volumes of records contained in contemporary data sets introduce potential redundancies, inconsistencies, and discrepancies, which can complicate the accurate identification of matching records. Additionally, the computational demands required necessitate efficient algorithms and scalable solutions to ensure precise and prompt results. Despite these challenges, advancements in algorithmic record linkage approaches [1] have facilitated more effective and efficient techniques specifically crafted for large data sets. By leveraging these technological innovations and adopting robust methodologies, organizations can navigate the complexities of record linkage in voluminous data, unlocking valuable insights and ensuring data-driven decision-making across various domains and applications.

A record linkage task of a pair of data sets comprises two distinct phases: blocking and matching. The former aims to group records in blocks that share some common attributes, termed as blocking keys, which are likely to indicate a potential match. Dates of birth, or zip

codes, or surname prefixes are commonly used as blocking keys to efficiently partition records into blocks. The purpose of blocking is to mitigate the quadratic complexity of the number of record pairs, each of which originate from different data sets, that are formulated in the blocks. During matching, these record pairs need to be resolved to determine their matching status. Approximate blocking techniques allow for more flexibility than the exact methods, because they consider similarity between attribute values instead of requiring exact matches.

Streaming record linkage allows for the continuous processing of data as it becomes available, enabling real-time decision-making and analysis. Attempting to match a query record against all previously processed records since the stream's inception, given certain time constraints, is both impractical and unrealistic [2,3]. Moreover, this approach may be ineffective, as real-time integration prioritizes newly arrived records to maintain temporal relevance. An alternative could be returning a complete result set using a persistent data store with relaxed time constraints. However, it might still be impractical to examine all potential matches. Ranking the returned results allows for more effective resource allocation by enabling the review of top-ranked matches while potentially disregarding lower-ranked results that are less likely to be matches.

* Corresponding author.

E-mail addresses: dkarapiperis@ihu.edu.gr (D. Karapiperis), c.tjortjis@ihu.edu.gr (C. Tjortjis), verykios@eap.gr (V.S. Verykios).

<https://doi.org/10.1016/j.is.2026.102692>

Received 4 April 2025; Received in revised form 13 January 2026; Accepted 14 January 2026

Available online 16 January 2026

0306-4379/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Several efforts have been made recently to devise efficient solutions for streaming record linkage. These works mostly rely on heuristics to accelerate the resolution process of the query records, such as the discontinuation of blocks that are considered oversized [4], or prioritizing in main memory the most recent records [5,6] or the most frequently accessed blocks [7]. The authors in [3] introduced a theoretical framework that evicts blocks from main memory based on both their frequency of access and recency of use. However, the primary emphasis of the aforementioned techniques is the acceleration of their query time, often overlooking the core function of blocking which is the accurate grouping of similar records based on their blocking keys.

For instance, most of these efforts for building their blocking structure rely on token blocking (or q -gram blocking) [8] or extended suffix array blocking [9], which assume common substrings of the blocking keys to create the blocks. However, these techniques tend to generate blocks containing a large number of non-matching records, as a match between substrings does not necessarily imply similarity between the corresponding blocking keys. Therefore, meta-blocking [10,11] is required to operate on top of the created blocks to reduce the number of record pairs that will be brought to the matching phase. However, this additional meta-blocking layer increases the overall time complexity, which results in longer query times.

In this work, we introduce DynaHash, a randomized record linkage blocking structure, which utilizes the MinHash [12] technique on the blocking keys of the records to create representations, which can, then, be efficiently blocked using the Hamming Locality-Sensitive Hashing blocking [13].

By assuming a pair of records, which consist of several strings, DynaHash extracts the blocking keys and uses the MinHash mechanism on these keys to generate a pair of compact vector representations, where the probability of their components to be equal, in the same positions, depends on the Jaccard similarity of the blocking keys. Hamming LSH blocks these vectors multiple times, using several hash tables, and provides both its theoretical guarantees of accuracy and sublinear running time with proper tuning of its parameters.

DynaHash consists of two components: (a) a persistent storage component, which is used to store the blocking structure of all records permanently, so as to provide complete results, and (b) an in-memory component that summarizes the persisted blocking structure, in order to provide partial results very fast with a certain probability. The number of slots of each block is bounded, which, in the case of a full block, necessitates the eviction of a record during the arrival of an incoming record. Another key aspect of our work is the drastic reduction of the space requirements of the blocking structure while preserving the formal guarantees of accuracy and performance. Specifically, DynaHash scans multiple *neighboring* blocks, in terms of the Hamming distances of their keys, in an attempt to find similar vectors which have been blocked therein. To the best of our knowledge, **this is the first work that utilizes Multi-Probe matching [14] in Hamming LSH** while yielding the same accuracy guarantees. We achieve a considerable reduction in the number of hash tables that are required, by a certain decrease factor. While this Multi-Probe technique has been explored for grouping in other contexts, such as the semantics-based approach presented by Azzalini et al. [15], our work is the first to formalize this approach for matching, yielding significant space reductions while maintaining theoretical accuracy guarantees.

Our experimental evaluation against three state-of-the-art methods on six real-world data sets indicates the superiority of DynaHash over its competitors and validates our theoretical claims. In Multi-Probe matching, we experimentally yield the same results using 67% fewer hash tables with the same set of parameters to ensure sublinear running time.

Our work makes the following contributions:

- It proposes a baseline randomized blocking structure, which identifies each matching record pair, based on the Jaccard similarity

of the corresponding blocking keys, with a certain probability in the Hamming space. It also creates refined blocks, contributing to the achievement of sublinear query times in the total number of blocked records.

- It introduces an in-memory data structure that summarizes the persisted blocking structure where the probability of identifying a record pair is inversely proportional to the number of evictions that occur in the corresponding blocks.
- It proposes a novel mechanism for ranking the results according to their similarity to the query record.
- It introduces Hamming LSH Multi-Probe matching by scanning additional blocks in the same hash table. This approach results in reducing the space requirements of the blocking structure while maintaining equivalent theoretical guarantees of accuracy.

The rest of this paper is structured as follows: Section 2 discusses the related work, while Section 3 provides the problem statement. Our proposed blocking structure is described in detail in Section 4, followed by the experimental evaluation in Section 5. Section 6 presents a detailed discussion of our findings and the practical trade-offs involved in tuning DynaHash. Finally, Section 7 offers our conclusions and directions for future work.

2. Related work

Numerous techniques for record linkage have been devised across various settings, including progressive [16–21], incremental [22], and topic-aware [2] contexts. Progressive record linkage concentrates on identifying matching record pairs at the outset of the linkage process. In contrast, incremental record linkage aims to identify matching record pairs efficiently when new records are introduced, utilizing existing data structures without redundant comparisons. These methods emphasize computational and memory efficiency for producing very fast accurate results. Persistent data stores typically underpin their corresponding in-memory data structures, which aim to enhance their generated result sets.

In the realms of online [23–25] settings, where extensive volumes of record are linked in near real-time, and more specifically in streaming record linkage, a limited number of works [3–7,26] have surfaced in the literature. Karapiperis et al. [6] proposed a method to address the substantial growth of blocks maintained in main memory. Specifically, they restrict the number of these blocks by adopting an eviction strategy to discard old blocks in favor of new ones originating from the stream. However, blocks experiencing short periods of inactivity may be promptly discarded despite their frequent access, which may potentially impact the accuracy of the returned results.

In [7], the authors propose a streaming algorithm that retains the most frequently accessed blocks in main memory to minimize secondary storage usage. This method suffers from scalability issues because it fails to account for the growth of records within each block, which may eventually necessitate the use of secondary storage.

Gazzari and Herschel in [4] advocate for a redundant block collection approach (TASK), discontinuing blocks exceeding a user-defined threshold and maintaining an in-memory quarantine list for discarding immediately records that target discontinued blocks. However, solely setting a maximum block size without using any other quality criteria may compromise accuracy.

Araujo et al. [5] propose a schema-agnostic approach (TIME) employing *meta-blocking* [10,11] to optimize block structures and discard redundant comparisons. Their method employs a time window technique to manage large volumes of records, prioritizing recent records in main memory to reduce memory utilization. However, the definition of this time window is not clear; a small time window may negatively affect recall, while a large time window may lead to a substantial increase in the number of blocks and records, risking memory overflow.

Table 1
Qualitative comparison of streaming record linkage methods.

Characteristic	DynaHash	TASK	TIME	SPK
Primary Goal	Balanced Speed & Recall	Parallelism	Schema-Agnostic	Simplicity & Strength
Blocking Core	Randomized (MinHash/LSH)	Token-based	Token-based	TF/IDF Weighted Tokens
Query Complexity	Sub-linear (persistent), Constant (in-memory)	Data-dependent	Constant (in window)	Data-dependent
Memory Management	Bounded in-memory summary	Discontinues oversized blocks	Fixed time-window	Relies on OS Caching
Accuracy Guarantees	Probabilistic Guarantees	Heuristic-based	Heuristic-based	Heuristic-based
Result Ranking	Native Support	Not Supported	Not Supported	Not Supported
Space Efficiency	Good (Excellent with Multi-Probe)	Poor	Acceptable	Acceptable

The authors in [27] present Sparkly (SPK) which has been developed on top of Apache Lucene.¹ Sparkly tokenizes each record using its 3-grams to index them with Lucene. For each query record, Sparkly searches for its top- K similar indexed records, using a suitably adapted version of the Okapi BM25 similarity function, which relies on TF/IDF weights. Sparkly requires the user to specify the number K of the top neighbors to retrieve for each query record. This requirement can negatively impact precision and recall, as it is challenging to estimate in advance the appropriate number of matching records for each query.

Several recent studies, such as [28–31], have utilized pre-trained Language Models (LM) for record linkage tasks. The techniques introduced in these studies initially leverage the LM's to convert the input records into dense embedding vectors, and subsequently, identify candidate record pairs by performing K nearest neighbors search based on a similarity metric. However, their effectiveness in streaming settings remains unexplored. Further research should concentrate on strategies to minimize the time needed for adaptation or fine-tuning to the data sets at hand. Applying an LM to data sets with distinct patterns without any fine-tuning, may generate inaccurate results due to the model's constrained comprehension of the target data.

The importance of fairness in record linkage is emphasized in [32, 33]. The former presents a novel workflow designed to address fairness within streaming data contexts, while the latter introduces fairness evaluation measures to mitigate biases inherent in record linkage systems.

Table 1 provides a qualitative comparison of DynaHash against its main competitors used as baseline methods in Section 5. DynaHash uniquely offers formal probabilistic guarantees on accuracy and sub-linear or constant query times through its randomized blocking structure. Furthermore, its native support for result ranking and the significant space savings from Multi-Probe matching fill a critical void for a solution that is simultaneously fast, accurate, and resource-efficient in streaming environments.

3. Problem statement

We assume a continuous stream of records which originate from two different data sets A and B . We utilize a blocking structure and a set of hash functions that are used to derive the blocking keys from each record in order to insert these records into the corresponding blocks. The core problems addressed in this study can be summarized as follows:

Problem Definition 1. Given a query record $q \in B$ (or A), find these records in A (or B) whose similarity of blocking keys meets threshold θ in a metric space using $\Theta(\sqrt{n})$ time in the total number n of blocked records.

Due to the very large number of records, we assume an in-memory and a persistent blocking structure, where the former summarizes the latter. The number of slots² of each block of the in-memory blocking structure is bounded by an integer constant w . If a block is fully occupied, then one of its records is evicted to give space to an incoming record. We also assume that the persistent blocking structure is unbounded.

Problem Definition 2. Design an eviction policy where the probability of a record surviving successive evictions decays over time. This ensures that the in-memory structure prioritizes temporal relevance by making space for fresh records, while providing a predictable and graceful degradation of accessibility for older records.

In the following, we introduce DynaHash, which implements the solutions for the problems posed.

4. DynaHash

The operation of DynaHash relies on two blocking structures: (a) one persistent and (b) one in-memory. The former is used to store all records processed in order to provide complete results, while the latter summarizes the complete blocking structure to resolve queries very fast and provide partial results.

4.1. Creation of the MinHash vectors

We combine the MinHash technique [12] and the Hamming distance in order to embed our blocking keys into a metric space where an LSH mechanism can efficiently identify the similar keys, according to a threshold, with theoretical guarantees. This allows us to form blocks whose records may potentially formulate matching pairs.

MinHash estimates how similar two sets are according to the Jaccard metric, by applying a hash function to the elements of a set to map these to integers. Subsequently, it picks the minimum integer as the result of its operation. The probability of the MinHash values of two sets to be the same is equal to the Jaccard similarity of these sets. We implement the MinHash technique using MurmurHash3,³ which is a non-cryptographic hash function, to map the q -grams⁴ of a string

² A slot corresponds to a record.

³ <https://github.com/aappleby/smhasher/blob/master/src/MurmurHash3.cpp>

⁴ The set of q -grams of a record consists of its substrings of length q .

¹ <https://lucene.apache.org>

to numerical hashes, in a well-distributed manner, and return the minimum of these hashes. Two sets of q -grams that correspond to a pair of strings is considered as similar if their Jaccard similarity meets threshold θ . The authors in [34] present a method on how one can associate edit distance with Jaccard similarity to derive the threshold in the Jaccard space.

DynaHash generates hash keys based on a blocking key, which is a plain string or a number of concatenated such strings. Although, we rely on the theoretical framework of [26] to derive the optimal number of hashes, we adopt a different method. Specifically, instead of using a single hash to play the role of the key, we concatenate m hashes generated for a blocking key in order to create its MinHash vector. This number m is derived according to the concept of *collisions* described in [26]. A collision occurs when a pair of MinHash vectors exhibits the same MinHash value at the same component. In summary, if we assume a pair of MinHash vectors, we expect a certain number of collisions which depend on their Jaccard similarity. We ensure that any deviation from the expected number of collisions remains within an allowable error margin, with a specified confidence, by repeating the MinHash technique m times. DynaHash uses these MinHash vectors to form the MinHash keys, which essentially specify the blocks to which these vectors are stored, as the following section describes.

4.2. The blocking phase

The Hamming distance between a pair of similar MinHash vectors is equal to the number of components in which they differ. Since we have bounded the number of their collisions, we have also bounded the number of their different components. A metric that is capable of quantifying formally the distance between a pair of MinHash vectors is the Hamming distance. Thus, Hamming LSH emerges as an appealing mechanism for implementing the blocking infrastructure. Hamming LSH relies on L independent hash tables, where each such hash table consists of key-bucket pairs that play the role of blocks. A bucket hosts a list of Id 's and MinHash vectors, each of which refers to a record. For each hash table, DynaHash maintains a list of k integers which have been chosen randomly and uniformly, with repetition, from the interval $[0, m - 1]$. Each list is used to pick the corresponding components of a MinHash vector, which is then concatenated to formulate the MinHash keys of that vector for each hash table. If a key already exists in a hash table, then the Id of the respective record is inserted into the corresponding bucket, otherwise a new block is created.

Intuitively, the smaller the Hamming distance γ of a pair of MinHash vectors is, the higher the probability of that pair to be blocked into at least one hash table, which is:

$$p_k = \left(1 - \frac{\gamma}{m}\right)^k. \quad (1)$$

Since, the expected number of collisions for a similar MinHash vector pair is at least $m\theta$, we translate θ into the Hamming distance threshold t as:

$$t = m - m\theta = [(1 - \theta)m], \quad (2)$$

which essentially expresses the maximum allowable number of components that a pair of MinHash vectors may differ in order to be considered as similar in the Hamming space. For the Hamming LSH scheme, the probability of any component, with the same index, chosen from a pair of MinHash vectors, whose Hamming distance is at most t , is given by $1 - \frac{t}{m} = \theta$, using Eq. (2). Therefore, if we assume the collision of the MinHash keys of a pair of similar MinHash vectors as a random trial, we need to calculate the required number L of these trials, which will be implemented as L hash tables. Thus, we need to ensure that a similar pair will collide in at least one out of these trials with a certain probability. Andoni and Indyk [13] calculate this required number of trials as:

$$L = \left\lceil \frac{\ln(\delta)}{\ln(1 - \theta^k)} \right\rceil, \quad (3)$$

where each similar MinHash vector pair is returned with probability at least $1 - \delta$. Their analysis is based on the fact that the probability of a similar MinHash vector pair **not** colliding in at least one trial should be $(1 - \theta^k)^L \leq \delta$, where they solve for L . The confidence parameter δ is usually set to $\delta \leq 0.1$ to magnify the success probability. A pair might collide several times, whose expected value is $L\theta^k$, depending on their similarity.

These guarantees hold for any similar pair, whose Hamming distance is at most t , because finding pairs that correspond to smaller distances than t , requires fewer hash tables for blocking than L .

Parameter k is set empirically as a function of the underlying data set in order to minimize the time for resolving queries [35] since the correctness of the scheme is guaranteed by adjusting L accordingly. In the next section, we demonstrate the choice of k by randomly selecting a small number of records to compute the average query time by varying k .

By increasing k , L grows considerably, by assuming that δ and θ remain constant, but the generated blocks are more refined and selective in terms of the blocked vectors. While this block refinement leads to fewer candidate pairs and smaller block sizes, it requires a larger number of hash tables to be generated, thus increasing the overall storage requirements. DynaHash follows a hybrid model that uses two blocking structures: The persistent structure DB and the in-memory structure called T . The basic difference between these structures is the chosen value of the length of their MinHash keys. The former uses k , which should be a strict value to generate more refined blocks, while the latter uses a lower, and consequently, looser value symbolized by ϕ , where $\phi < k$.

Specifically, DynaHash extracts a prefix of ϕ positions of the MinHash keys of DB to generate each key for T . By doing so, each block of T is a generalized version of a group of blocks of DB , because it essentially accommodates into one block records that have been inserted into multiple blocks of DB . Fig. 1 illustrates the generalization of two k -length blocks from DB by a single ϕ -length block in T .

This generalization may populate the blocks of T with a large number of records, which would potentially result in a memory overhead and in prolonged running times during the resolution of the query records. To mitigate these side effects, we bound the number of slots of each block of T by an integer constant w , and implement a simple strategy to insert a new incoming MinHash vector into a block, which is fully occupied. DynaHash randomly and uniformly evicts an old vector found in a fully occupied block to empty a slot for the incoming vector. Therefore, each block of T summarizes a number of blocks of DB .

The redundancy of the LSH mechanism is used as a safeguard because a query vector may collide with the evicted vector in a block of another hash table. This strategy allows DynaHash to provide guarantees for the lifetime of each vector in T throughout $s \times w$ of evictions, where s is a scaling factor, as the following lemma suggests.

Lemma 1. *The probability of a vector to remain in a block after at most $s \times w$ evictions is at least e^{-s} .*

Proof. First, we assume the worst-case scenario which is an incoming record hashing to all of the L_ϕ blocks that a certain MinHash vector resides. The probability P that this vector is evicted across all L_ϕ hash tables is $P = \frac{1}{w^{L_\phi}}$. Therefore, the probability P' of the complement event, which is the vector remaining in at least one hash table is $P' = 1 - \frac{1}{w^{L_\phi}}$. On the other hand, if an incoming record hashes to a single block that contains that vector, then $P' = 1 - \frac{1}{w}$. Hence, the probability of a vector remaining after at most w independent evictions lies within the following range:

$$\left(1 - \frac{1}{w}\right)^w \leq P' \leq \left(1 - \frac{1}{w^{L_\phi}}\right)^w \Rightarrow \frac{1}{e} \leq P' \leq e^{-w^{1-L_\phi}}, \quad (4)$$

where we have used $1 - \frac{1}{w} \approx e^{-\frac{1}{w}}$ by assuming $w \geq 500$. Thus, after at most $2 \times w$ evictions, this probability would be at least $\frac{1}{e^2}$ and so forth, which concludes the proof. $\square$

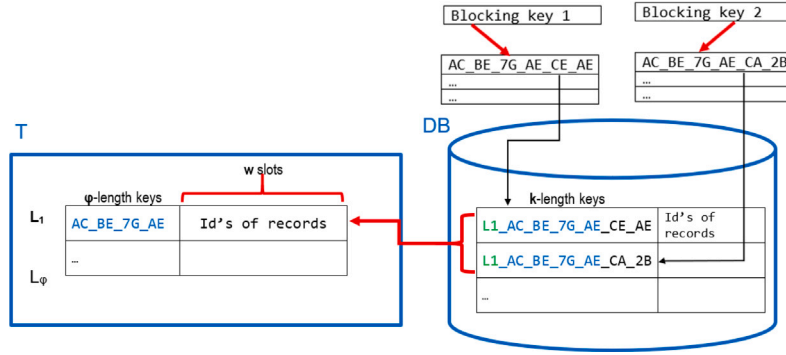


Fig. 1. The main components of DynaHash, where T generalizes DB . In this snapshot, the two k -length blocks, whose keys are $AC_BE_7G_AE_CE_AE$ and $AC_BE_7G_AE_CA_2B$, from DB are generalized into a single ϕ -length block, with a single key $AC_BE_7G_AE$, in T . Component $L1$ denotes the index of the hash table in DB .

An incoming query vector will locate vector v within at least one block, where at most $s \times w$ evictions have occurred since the admission of v , with a probability that scales proportionally to e^{-s} weighted by p_ϕ (refer to Eq. (1)). The probability p_ϕ signifies the likelihood of a pair of vectors hashing to the same block, which depends on their Hamming distance. Therefore, the revised probability of formulating a similar vector pair in a block leads to the following lemma.

Lemma 2. *The probability P of formulating a similar MinHash vector pair after at most $s \times w$ evictions across all L_ϕ hash tables is:*

$$P \geq 1 - (1 - \theta^\phi \frac{1}{e^s})^{L_\phi}. \quad (5)$$

Proof. The probability of finding a similar MinHash vector pair in at least one hash table is at least $1 - (1 - \theta^\phi)^{L_\phi}$ [13]. Using T though, the probability of hashing two similar vector pairs to the same block of any hash table after at most $s \times w$ evictions is at least $\theta^\phi \frac{1}{e^s}$, which concludes the proof. $\square$

Probability P plays the role of the lower bound for finding all similar vector pairs, due to the use of thresholds θ and t , whose combination is depicted in Eq. (2).

As soon as no more blocks can be created due to main memory limits, DynaHash initializes T and then resumes its operation. We note that the blocks of DB are unbounded to maintain all MinHash vectors, and their corresponding records, seen so far, so as to be able to return a complete result, should such a requirement arise.

Algorithm 1 showcases the process of inserting a record r to the internal data structures of DynaHash. Initially, the blocking key is extracted from r (Line 1), which is used for creating its MinHash vector H in lines 3 and 4 using m MinHash functions. Then, DynaHash formulates its MinHash key for each hash table by picking k components of H as specified by list S_i (Lines 7 until 9). In Line 10, the index of the current hash table and key are concatenated to form the database key that will be used for the insertion of r and H into DB , as shown in Line 11. Finally, if the index of the current hash table is within L_ϕ , then key_ϕ is extracted from key_s in order to be used as the MinHash key for storing the record and its vector into T (Lines 13 and 14). The records are maintained into a single repository to which DynaHash refers, via their Id 's, to fetch them.

The space required for storing the blocks in DB and T is $\mathcal{O}(L \times n)$ and $\mathcal{O}(L_\phi \times w)$, respectively, where n denotes the total number of records. The total running time for blocking is $\mathcal{O}(L \times n)$.

4.3. The matching phase

During the matching phase, DynaHash extracts the blocking key of each query record and, then, creates its MinHash vector, which

Algorithm 1 Blocking record r using Hamming LSH.

Input: Record r , k , ϕ , samples S , L , and L_ϕ .

```

1:  $bk \leftarrow \text{extract\_bkey}(r)$ 
2:  $H \leftarrow []$ 
3: for  $j = 1$  to  $m$  do
4:    $H.append(\text{MinHash}(bk, j))$   $\triangleright$  A MinHash function, using
   seed  $j$ , is applied to the blocking key  $bk$ . The result is appended to
    $H$  in order to formulate its MinHash vector.
5: for  $i = 1$  to  $L$  do
6:    $key \leftarrow []$ 
7:   for  $j = 1$  to  $k$  do
8:      $ind \leftarrow S_i[j]$ 
9:      $key.append(H[ind])$   $\triangleright$  The MinHash key
   key is formulated by appending the components of  $H$  specified by
   list  $S_i$ , which is attached to the  $i$ -th hash table.
10:  $dbk \leftarrow \text{concat}(i, key)$ 
11:  $DB.insert(dbk, r, H)$   $\triangleright$  Record  $r$  and its MinHash vector  $H$ 
   are persisted to  $DB$ , where  $dbk$  plays the role of the corresponding
   database key.
12: if  $i < L_\phi$  then
13:    $key_\phi \leftarrow \text{extract}(key, 0, \phi)$ 
14:    $T[i][key_\phi] \leftarrow \text{upload}(r, H)$   $\triangleright$  Record  $r$ 
   and MinHash vector  $H$  are stored in  $T$  in main memory, where  $key_\phi$ 
   is the MinHash key. If there is no free slot in the respective block,
   then a randomly chosen vector is evicted to release a slot.

```

is used to derive the MinHash keys, using ϕ , for each hash table. Subsequently, DynaHash scans the hash tables of T to retrieve the respective MinHash vectors of each block and calculate the Hamming distances. The records that correspond to these vectors, which have been identified as potentially matching, may be optionally compared with the query record using any matching condition to refine their matching status, prior to engaging in a clerical review process.

In order to prevent repetitive distance calculations, a de-duplication mechanism may be used by utilizing an efficient structure (like a Bloom filter [36] or a simple hash map) for checking membership of each Id of a MinHash vector, which has been retrieved from T .

In highly demanding settings, where the completeness of the results is of utmost importance, then DB may be queried to enhance the results with records which have been evicted from T . For this task, the keys that are generated, in order to access the corresponding blocks, use the additional $k - \phi$ positions, as described in Section 4.2.

Algorithm 2 illustrates the resolution of a query record q using T . First, DynaHash extracts the blocking key of q (Line 1) in order to create its MinHash vector H (Lines 3 and 4). Following this, for each hash table of T , DynaHash formulates the respective MinHash key (Lines 7

until 9) in order to retrieve the corresponding MinHash vectors, as Line 10 shows. DynaHash, then, calculates the Hamming distances between these vectors and H (Line 13), whose Id 's are added to the set of matching records mp if the Hamming distances meet threshold t (Line 14).

Algorithm 2 Resolving a query record q using T .

Input: Query record q , T , the Hamming threshold t , ϕ , samples S , L_ϕ , and an empty set mp for storing the Id 's of the matching records.

Output: The set mp that contains the Id 's of the matching records to q .

```

1:  $bk \leftarrow \text{extract\_bkey}(q)$ 
2:  $H \leftarrow []$ 
3: for  $j = 1$  to  $m$  do
4:    $H.append(\text{MinHash}(bk, j))$ 
5: for  $i = 1$  to  $L_\phi$  do
6:    $key_\phi \leftarrow []$ 
7:   for  $j = 1$  to  $\phi$  do
8:      $ind \leftarrow S_i[j]$ 
9:      $key_\phi.append(H[ind])$ 
10:   $vs \leftarrow T[i][key_\phi]$   $\triangleright$  DynaHash retrieves the MinHash vectors
    from block  $key_\phi$  of the  $i$ -th hash table of  $T$ .
11:  for each  $G \in vs$  do
12:    if  $G.Id \notin mp$  then
13:      if  $D(H, G) \leq t$  then  $\triangleright$  If the Hamming
        distance between the MinHash vectors  $G$  and  $H$  meets  $t$ , then the
         $Id$  of  $G$  (the  $Id$  of the corresponding record can be used) is added
        to the set of the potentially matching records for  $q$ .
14:       $mp.add(G.Id)$ 

```

The query time is data dependent, but by setting properly parameters ϕ and k , which affect L_ϕ and L , respectively, usually results in $\Theta(\sqrt{n})$ time in the total number of records that are blocked, which is experimentally demonstrated in Section 5.

4.4. Multi-probe matching

Instead of scanning a single block in a hash table, Multi-Probe LSH [14] looks in several blocks in the same hash table, whose keys exhibit slight differences. This increases the chance of finding similar vectors that might not be in the exact same block, but exist in nearby blocks. Multi-probe LSH systematically checks blocks that are likely to contain similar vectors based on the hash function used. Thus far, Multi-Probe LSH and its variants are primarily associated with the Euclidean LSH, which is designed for data sets where the distance measure is based on the Euclidean distance. In our scheme, we investigate the potential to probe several blocks in a single hash table, where the Hamming distances from the corresponding MinHash keys are small. This Multi-Probing increases the likelihood of finding similar MinHash vectors in these neighboring blocks, thereby decreasing the space requirements by utilizing fewer hash tables while maintaining the same level of recall.

Apart from the *main* block that should be anyway scanned, we additionally locate these blocks, whose keys differ by ω MinHash values. Given the small number k of components⁵ of the MinHash keys, we use a Burkhard-Keller Tree, or BK-Tree for short [37], which is a simple space-partitioning data structure that works in discrete metric spaces. Specifically, a BK-Tree is attached to a hash table in order to return its keys within a specified Hamming distance to a MinHash key in $\Theta(\log b)$ time, where b is the total number of MinHash keys in that hash table. In doing so, we essentially relax k by using fewer

⁵ Given this small number of components (< 10), the problems typically associated with high-dimensional data (known as the curse of dimensionality) are avoided.

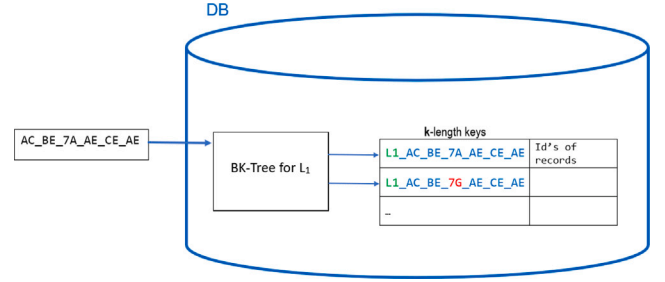


Fig. 2. In Multi-Probe matching, in addition to the main block that a query record hashes into, neighboring blocks are also scanned. These neighbors are blocks within a specified Hamming distance (ω). In this example, besides the primary block $AC_BE_7A_AE_CE_AE$, the block $AC_BE_7G_AE_CE_AE$ is also scanned because their Hamming distance falls within ω ($\omega = 1$). A BK-Tree is used to efficiently find these neighboring blocks. Component $L1$ denotes the index of the hash table in DB.

hashes, namely $k - \omega$, to locate blocks, which have already been refined during blocking due to the use of k , which is stricter than ω . Fig. 2 illustrates this Multi-Probe approach for $\omega = 1$, where, in addition to the main block $AC_BE_7A_AE_CE_AE$ of the first hash table (L_1) that the query record hashes into, $AC_BE_7G_AE_CE_AE$ is also scanned. The Hamming distance between these blocks is 1, which is within threshold ω .

The number of hash tables required based on this Multi-Probe approach is revised according to the following lemma.

Lemma 3. The factor of decrease α in the number of hash tables required for Multi-Probe matching compared to the baseline is approximately:

$$\alpha = \frac{1 - \theta^{-(\omega+1)}}{1 - \theta^{-1}}. \quad (6)$$

Proof. A collision of a vector pair is mutually exclusive in these blocks, since only a single collision of a vector pair can occur in a certain hash table. Thus, the success probability of a vector pair to collide in a hash table is:

$$p_\omega = \sum_{i=0}^{\omega} \theta^{k-i} = \theta^k \left(1 + \frac{1}{\theta} + \frac{1}{\theta^2} + \dots + \frac{1}{\theta^\omega} \right) = \frac{1 - \theta^{-(\omega+1)}}{1 - \theta^{-1}} \theta^k = \alpha \theta^k, \quad (7)$$

where $\alpha > 0$ given θ ranges in $(0, 1)$ and ω is a positive integer.

Hence, the success probability of a similar vector pair to collide in at least one hash table out of L is $1 - (1 - p_\omega)^L$, based on which the required number of hash tables for the probing scheme, given Eq. (3) and the approximation $1 - \alpha^k \leq e^{-\alpha^k}$, becomes:

$$L_\omega = \left\lceil \frac{\ln(\delta)}{\ln(1 - \alpha \theta^k)} \right\rceil \approx \left\lceil \frac{\ln(\delta)}{-\alpha \theta^k} \right\rceil, \quad (8)$$

In Eq. (3), L can be simplified to:

$$L = \left\lceil \frac{\ln(\delta)}{-\theta^k} \right\rceil, \quad (9)$$

using the same approximation. In turn, dividing both sides with α :

$$\frac{L}{\alpha} = \left\lceil \frac{\frac{\ln(\delta)}{-\theta^k}}{\alpha} \right\rceil = \left\lceil \frac{\ln(\delta)}{-\alpha \theta^k} \right\rceil = L_\omega, \quad (10)$$

we arrive at L_ω . Therefore, given fixed θ , δ , and k , as ω increases in Eq. (7), L_ω decreases by approximately a factor of α , namely $L_\omega \approx \left\lceil \frac{L}{\alpha} \right\rceil$, which concludes the proof. $\square$

In order to maintain the guarantees of accuracy, described in Section 4.2, the total number of blocks that will be scanned throughout

all hash tables should be equal to L_ω . Therefore, DynaHash creates L_ω hash tables during blocking, which is the maximum number of hash tables that could be used during matching by assuming that at least one block of each of these hash tables is used.

The main difference between the Multi-Probe and the baseline DynaHash is the length of the MinHash keys stored in *DB*. In the Multi-Probe scheme, the length of the MinHash keys is k , but the number of hash tables is L_ω . In contrast, the baseline uses ϕ -length keys in *T* and k -length keys in *DB*, along with the corresponding required numbers of hash tables.

Algorithm 3 illustrates the matching process using Multi-Probing. For each of the L_ω hash tables, a *main_key* is generated for the query record (Line 10). The corresponding BK-Tree, which is attached to that hash table, is then queried to retrieve all keys that are within a Hamming distance of ω from the *main_key* (Line 12). Variable *blocks* stores the total number of blocks of *DB*, which are signified by these MinHash keys, that will be probed throughout the whole matching process (Line 13). Then, these blocks are scanned to find the potentially matching records to the query record q (Lines 14 until 19). Line 20 determines the termination of the matching process after probing L_ω blocks.

Thus, in an already created blocking structure, utilizing smaller MinHash keys (in terms of the number of their components) to locate the blocks, DynaHash achieves: (a) the maintenance of block refinement, which means fast resolution of the query records, and (b) surprisingly high recall rates. Using Multi-Probe matching, DynaHash essentially provides the same accuracy guarantees but with much lower space requirements than the baseline due to the utilization of the additional neighboring blocks, except for the main one.

The query time depends on ω , which decelerates the performance of each BK-Tree as ω increases, since more MinHash keys need to be examined. The average query performance is $\Theta(\log(n))$ in the number of MinHash keys b of each hash table. Consequently, the total query time is $\Theta(\log(b) + \sqrt{n})$ in the total number of records n that are blocked. The space required for storing the blocks in *DB* is $\mathcal{O}(L_\omega \times (n + b))$, where $\mathcal{O}(b)$ is the space complexity of each BK-Tree.

Algorithm 3 Resolving a query record q with Multi-Probing using *DB*.

Input: Query record q , *DB*, the Hamming threshold ω , the number L_ω of hash tables that will be used, the Hamming threshold t , samples S , and an empty set *mp* for storing the *Id*'s of the matching records.
Output: The set *mp* that contains the *Id*'s of the matching records to q .

```

1:  $bk \leftarrow \text{extract\_bkey}(q)$ 
2:  $H \leftarrow []$ 
3: for  $j = 1$  to  $m$  do
4:    $H.append(\text{MinHash}(bk, j))$ 
5:  $blocks \leftarrow 0$ 
6: for  $i = 1$  to  $L_\omega$  do
7:    $main\_key \leftarrow []$ 
8:   for  $j = 1$  to  $k$  do
9:      $ind \leftarrow S_i[j]$ 
10:     $main\_key.append(H[ind])$ 
11:    $main\_dbk \leftarrow \text{concat}(i, main\_key)$ 
12:    $keys \leftarrow \text{tree}[i].get(main\_key, \omega)$ 
13:    $blocks \leftarrow blocks + keys.length$ 
14:   for each  $k \in keys$  do
15:      $vs \leftarrow DB.get(k)$ 
16:     for each  $G \in vs$  do
17:       if  $G.Id \notin mp$  then
18:         if  $D(H, G) \leq t$  then
19:            $mp.add(G.Id)$ 
20:   if  $blocks \geq L_\omega$  then
21:     break

```

4.5. Ranking the results by adjusting the Jaccard threshold

Instead of processing the entire blocking structure, which includes L hash tables, adjusting the Jaccard threshold at a specified position within the interval $(0, \dots, 1)$, DynaHash would need to scan fewer hash tables than the total available, due to the use of a stricter threshold. This adaptation can significantly speed up the matching step, making it more scalable, especially with very large data sets. Furthermore, this prioritization optimizes the use of computational resources by reducing the workload to only scanning the required number of hash tables, which depends on the targeted level of similarity.

We initially choose a new Jaccard threshold θ' , where $\theta' \geq \theta$, based on which the similarity ranks will be created. Specifically, DynaHash gradually relaxes the similarity threshold in the interval $[\theta', 1)$ starting from a value near 1, e.g., 0.95, using a small step st , e.g., $st = 0.05$, which moves towards θ' . Hence, this gradual relaxation of threshold creates similarity intervals, such as $[0.95, 1), [0.90, 0.95), \dots, [\theta', \theta' + st)$. Each such interval essentially specifies the similarity rank to which each returned vector pair belongs. The higher the rank of a vector pair is, the higher their underlying similarity is. Additionally, for each lower rank, more hash tables are utilized to provide the guarantees of accuracy, whose number will always be less than L . The output of this process is a list of vectors similar to a query, grouped by their similarity rank.

Algorithm 4 illustrates the ranking process using *DB*. We note that the algorithm is also applicable to *T*, where ϕ should be used instead of k . We symbolize by θ_0 the sliding value of the Jaccard threshold, whose initial value should be as close to 1 as possible, because we aim to find these records that exhibit the highest similarity early on. The similarity step st will gradually drive the matching process towards reaching θ' . If $\theta' = \theta$, then the results of the matching process will be identical to the complete matching process, except that the records returned will also be ranked based on their similarity to q .

In Line 5, DynaHash generates a series of Hamming thresholds by relaxing the corresponding Jaccard thresholds within the interval $[\theta', 1)$. This effectively creates a sequence of bins, which correspond to ranks $[\theta_0, 1), [\theta_0 - st, \theta_0), \dots, [\theta', \theta' + st)$, whose endpoints are translated into the Hamming space.

The algorithm operates in a main loop using the newly calculated values of θ_0 in each iteration. For each such θ_0 , a pair of indices L_1 and L_2 is calculated (Lines 6 and 8), which specify the starting and the ending hash table, respectively, that should be scanned in order to provide the guarantees of accuracy described in Section 4.2. The higher the value of θ_0 is, the less the number of the hash tables that will be used. In Lines 14 and 15, DynaHash concatenates the index of the current hash table and *key* to form the database key that will be used for retrieving the corresponding MinHash vectors from *DB*. Each distance calculated between these vectors and the query vector H is discretized to determine the rank to which the distance belongs. This discretization effectively results in ranking the corresponding vector pairs according to their Hamming distances.

After the completion of matching for a pair of indices L_1 and L_2 , DynaHash recalculates L_1 and L_2 (Lines 21 and 8), as well as the new θ_0 (Line 22). In Line 23, if θ_0 falls below θ' , then the whole process terminates.

5. Experimental evaluation

We evaluated DynaHash, referred to as DH, against TASK [4], TIME [5], and SPK [27]. Although SPK has not been designed to operate in real-time settings, its query times are exceptional due to the utilization of Apache Lucene.

We assess the recall and precision rates of DH and its competitors, along with the time required to resolve query records. Recall represents the proportion of the correctly identified matching record pairs out of the total number of existing pairs. Precision denotes the ratio of correctly identified matching pairs to the total number of pairs classified

Algorithm 4 Ranking the similar MinHash vectors to a query record q using DB .

Input: Query record q , DB , k , the Hamming threshold t , the initial Jaccard threshold θ_0 , θ' , step st , samples S , and an empty set mp for storing the Id 's of the matching records.

Output: The set of $bins$ that contain the Id 's of the matching records to q ranked according to their similarity.

```

1:  $bk \leftarrow \text{extract\_bkey}(q)$ 
2:  $H \leftarrow []$ 
3: for  $j = 1$  to  $m$  do
4:    $H.append(\text{MinHash}(bk, j))$ 
5:  $ranks \leftarrow \text{create\_ranks}(\theta_0, \theta', st)$ 
6:  $L_1 = 1$ 
7: while True do
8:    $L_2 = \left\lceil \frac{\ln(\delta)}{\ln(1-\theta_0^k)} \right\rceil$ 
9:   for  $i = L_1$  to  $L_2$  do
10:     $key \leftarrow []$ 
11:    for  $j = 1$  to  $k$  do
12:       $ind \leftarrow S_i[j]$ 
13:       $key.append(H[ind])$ 
14:     $dbk \leftarrow \text{concat}(i, key)$ 
15:     $vs \leftarrow DB.get(dbk)$ 
16:    for each  $G \in vs$  do
17:      if  $G.Id \notin mp$  then
18:        if  $D(H, G) \leq t$  then
19:           $rank \leftarrow \text{discretize}(D(H, G))$ 
20:           $ranks[rank].add(G.Id)$  ▷ Each
            distance calculated between these vectors and the query vector  $H$ 
            is discretized to assign it to a specific rank.
21:    $L_1 = L_2 + 1$ 
22:    $\theta_0 = \theta_0 - st$ 
23:   if  $\theta_0 < \theta'$  then
24:     break

```

Table 2
Characteristics of the data sets used.

	$ A $	$ B $	$ G $	Blocking keys
SCD	64K	2.6K	5K	authors+title
ACD	2.2K	2.6K	2.2K	authors+title
APB	1K	1K	1K	name of the product
OAG	1M	1M	300K	authors+title
NCVR	1M	1M	1M	last name+PO Box
DBLP	8M	8M	8M	authors+title

A and B denote the data sets to be linked. G is the gold standard.

as matches. We also use the F1 score which combines precision and recall into a single value, namely $\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$, providing a balanced assessment of the performance of a method. Recall, precision, and F1 score yield values within the range of $[0, 1]$, with higher scores indicating superior performance.

5.1. Data sets and configuration

We have used six real-world data sets, namely (a) NCVR,⁶ which includes a registry of voters, (b) DBLP,⁷ which comprises bibliographic records⁸ from the domain of computer science, downloaded in November 2023 in terms of an XML dump file (c) Open Academic Graph, (OAG) [38] which includes details of multidisciplinary academic papers from Microsoft Academic Graph which has been paired with Aminer,

(d) Scholar-DBLP (SCD) and (e) ACM-DBLP (ACD),⁹ which comprise bibliographic records, and (f) Apt-Buy (APB), Section 5.1 which includes e-commerce records. For NCVR and DBLP, each record from the original data sets was chosen with probability $\frac{1}{2}$, in order to generate 2 perturbed records to populate their counterpart data sets. Table 2 summarizes the characteristics of the data sets used throughout the experimental evaluation.

We performed the experiments using a virtual machine with 8 vCPUs (Intel Xeon), 48 GB of RAM, and a 500 GB SSD for storage. All algorithms, including DynaHash, were developed using Python 3.10 and executed in a single-threaded manner to ensure a fair comparison of their core logic without the influence of parallelization. We ran each experiment 50 times and plotted the average values as well as the standard deviations for the measurements obtained. DynaHash uses RocksDB,¹⁰ which is a simple, yet efficient, key-value storage library developed by Meta, as the underlying data store. RocksDB actively manages its own memory and disk usage. To ensure we properly evaluated the I/O performance of DB , we configured the RocksDB block cache to a conservative size (1 GB). This prevents the entire dataset from being cached in RocksDB's memory, forcing regular disk access and providing a more realistic characterization of the I/O performance that would be expected with datasets that vastly exceed available RAM. Each MinHash vector that corresponded to a composite¹¹ blocking key comprised $m = 116$ components. For creating the LSH structure, we fixed $\delta = 0.1$ and $\theta = 0.5$ and varied k from 4 until 7. Using these parameters, we blocked 100K randomly chosen records from DBLP in DB . Subsequently, Fig. 3(a) shows the query times as well as the number of the hash tables that each k yielded. We clearly observe the steep decrease of the query time for $k = \{4, 5, 6\}$ and its increase by setting $k > 6$. Therefore, we choose $k = 6$ to be the length of the MinHash keys in DB , because it minimizes the query time for this particular data set. We resumed this process for the rest of the data sets, which also highlighted the same value. For realizing T , we set $w = 500$ and $\phi = 4$, which yielded $L_\phi = 36$ hash tables, unless specified otherwise.

5.2. Experimental results

In this section, we present the results of a comprehensive set of experiments designed to evaluate the performance of DynaHash. The following subsections detail our findings on scalability and query performance, ranked matching, Multi-Probe matching, and finally, a comparative analysis in a simulated streaming environment.

5.2.1. Scalability and query performance

In the first set of experiments, we evaluated the speed at which DH resolved a query by scaling the underlying number of blocked records. We assumed both partial and complete results, which were resolved by T and DB , respectively. Fig. 3(b) displays the average query times and the number of records, which were brought in for distance computations by scaling the number of blocked records from DBLP starting from 100K until 8M records. We observe that T exhibits faster speed-ups than DB , due to the faster utilization of its hash tables. Furthermore, DB brings for comparisons nearly $\sqrt{n}$ records, excluding duplicates, which is a direct effect of properly tuning parameter k . On the other hand, we note that T uses a constant number of records for each query, which depends on w . This accounts for the constant query times exhibited for the cases of 5M and 8M blocked records.

⁹ https://old.dbs.uni-leipzig.de/research/projects/object_matching/benchmark_datasets_for_entity_resolution

¹⁰ <https://github.com/facebook/rocksdb>

¹¹ A composite blocking key may comprise more than one textual blocking keys.

⁶ <https://www.ncsbe.gov/results-data/voter-registration-data>

⁷ <https://dblp.org/xml>

⁸ We used the proceedings, inproceedings, and articles.

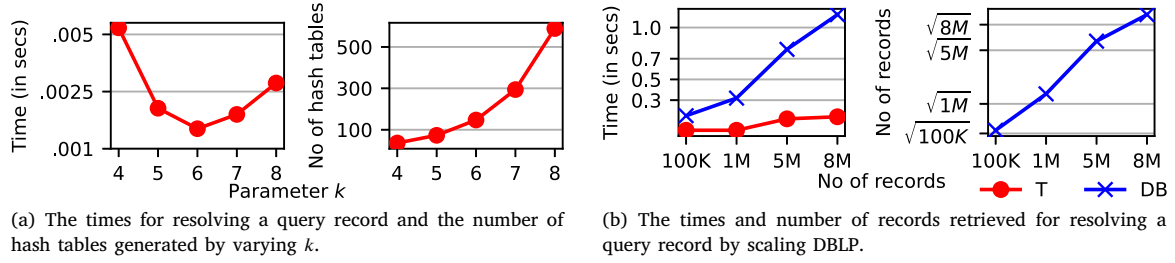


Fig. 3. Evaluating query time.

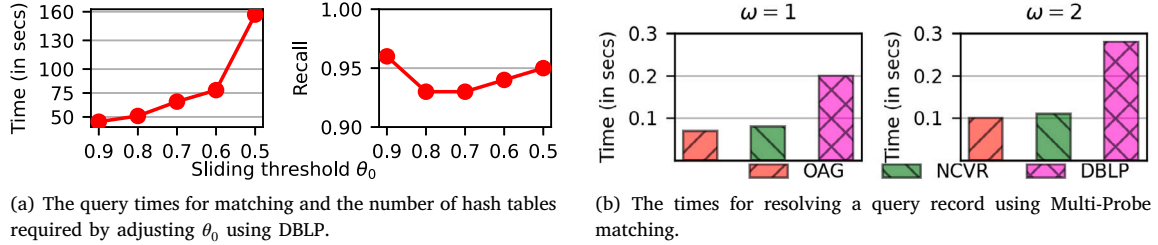


Fig. 4. Evaluating query time of the sliding threshold and Multi-Probe matching.

5.2.2. Ranked matching evaluation

Next, we utilized a constructed blocking structure of the NCVR data set, sampling 100,000 perturbed records to perform the ranked matching process. Fig. 4(a) illustrates the times required as the Jaccard similarity threshold slides from 0.9 down to $\theta' = \theta = 0.5$. We observe an almost linear relationship between the sliding threshold and time until $\theta' = 0.5$, where the time escalates significantly. This is undoubtedly due to the utilization of all the available 147 hash tables. The recall in each rank remains at high levels, specifically above 0.9, which means that at least 90% of record pairs, represented by the set of their q -grams, whose Jaccard similarity is at least θ_0 for each rank, are successfully identified in the Hamming space via their corresponding MinHash vectors. These rates verify the accuracy of our embedding scheme demonstrating its effectiveness in preserving similarity relationships and ensuring that similar pairs are correctly mapped and retrieved.

5.2.3. Multi-Probe matching performance

Fig. 4(b) shows the times for resolving a query record using Multi-Probe matching with the large data sets. By setting $\omega = 1$ and $\omega = 2$, the space requirements in DB are 67% and 86% lower than the baseline due to the reduced number of hash tables, which are $L_\omega = 48$ and $L_\omega = 20$, respectively. Although the recall rates are consistently high, averaging 0.9, the query times exhibit some variations, which are justified by ω . Specifically, setting $\omega = 2$, the query times are on average 33% longer due to the delay of the replies of the corresponding BK-Trees. We also stress that for a query using $\omega = 1$, an average of 3 blocks are used per hash table, which means that on average 16 hash tables out of 48 are eventually used. For $\omega = 2$, 16 blocks are utilized, implying that on average only 2 hash tables are used.

We limited our experiment to $\omega = 2$ to illustrate the clear trade-off involved. As ω increases, the number of required hash tables (L_ω) decreases significantly, leading to substantial space savings (86% for $\omega = 2$). However, the query time increases because the BK-Tree must explore a much larger search space to find all keys within the given Hamming radius. For $\omega > 2$, we observed that the increase in query time began to outweigh the benefits of space savings for these datasets, suggesting that a small ω (e.g., 1 or 2) strikes the best balance between reducing storage and maintaining fast query performance.

5.2.4. Comparative analysis on small datasets

We, then, evaluated the capabilities of DH, with DB, and its competitors using the small data sets. Figs. 5(a), 5(b), and 5(c) shows the recall, precision, F1-score rates, as well as the time needed to resolve a query record for SCD, ACD, and APB, respectively. All methods exhibit an outstanding performance in terms of recall, approaching or exceeding 0.9. The SCD dataset is non-duplicated¹² and features one-to-many matching relationships between its two sources. Due to this multiplicity, we specified $top - K = 20$ for SPK to ensure that the results would contain as many true positives as possible. However, this led in many cases to return superfluous records, which deteriorated the precision rates. On the other hand, for ACD, which is a fully deduplicated data set, by setting $top - K = 1$, SPK exhibited high levels of precision. DH constantly outperformed the rest of the methods in terms of recall reaching almost on average 0.97 with a very small standard deviation ± 0.02 as the error bars show. The precision is a little lower than SCD due to the relaxed threshold, which allowed the formulation of more false positives.

For APB, we had to adjust the parameters due to the very low similarities between the matching pairs. We relaxed θ to 0.45 and also increased the number of hash tables by setting $\delta = 0.001$ so as to give more chances to these matching pairs to be formulated. These adjustments increased recall to almost 0.86, but also decreased precision which was on average around 0.45 as Fig. 5(c) indicates. All methods suffered not only from lower precision but also from lower recall, with an average reduction of 8%.

5.2.5. Comparative analysis in streaming simulation on large datasets

To evaluate the performance in a streaming context, we simulate the continuous arrival of data. Our simulation proceeds in timesteps. At each timestep, a batch of 5K records (sampled from the source datasets A and B) is processed. For each record in the incoming batch, we treat it as a **query** against the blocking structure, which contains all records processed in previous timesteps. The record is first used to find matches, and then it is inserted into the blocking structure, causing the total number of blocked records (n) to grow over time. This methodology allows us to evaluate the query time and accuracy

¹² The non-duplicated data sets are also quoted as *dirty* in the literature.

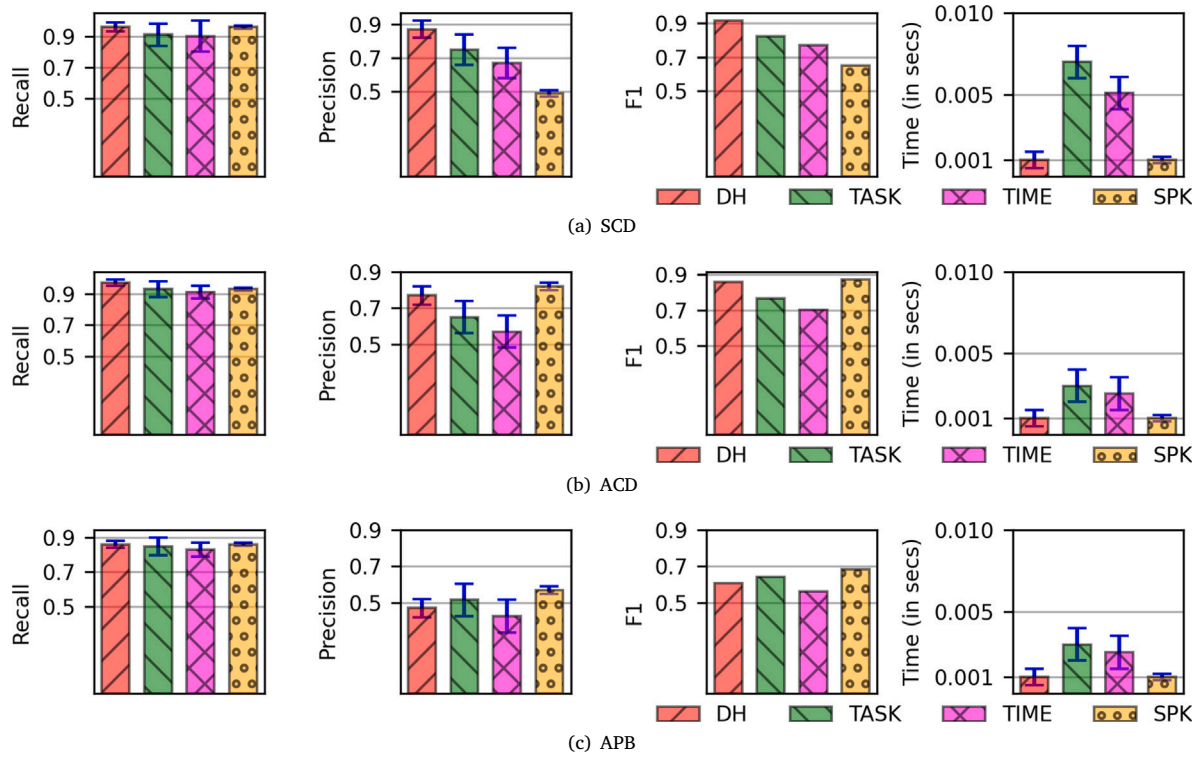


Fig. 5. Evaluating the performance of DH and its competitors using SCD, ACD, and APB.

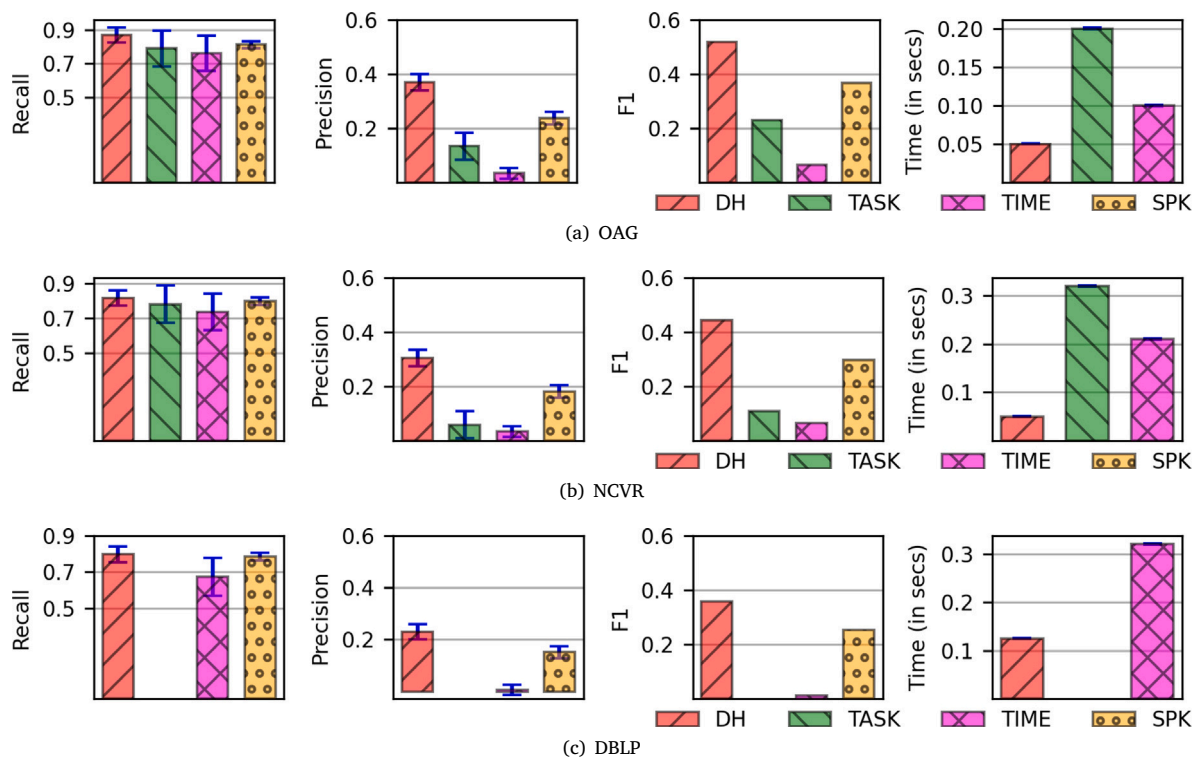


Fig. 6. Evaluating the performance of DH and its competitors OAG, NCVR, and DBLP.

of each method as the underlying data volume scales, which is a main characteristic of a streaming scenario. We chose a batch size of 5K records as a pragmatic compromise between simulating a continuous flow and keeping the total experiment runtime manageable. The underlying logic of DynaHash, processing each incoming record as a query before insertion, is identical regardless of batch size.

For OAG, whose results are shown in Fig. 6(a), we notice the superiority of DH which outperforms SPK in terms of recall and precision by 6.8% and 58.2%, respectively. For SPK, we had set $top - K = 3$ in order to increase its recall, however, it negatively affected its precision. TASK and TIME initially utilize redundant blocking structures to group together records that exhibit the same token. Subsequently, meta-blocking operates on top of these structures to reduce the number of the record pairs that will be compared based on several heuristics. This excessive number of records found in each block results in notable recall rates, with some fluctuations, at the expense of low precision, which falls for both methods below 15%. Regarding query times, DH demonstrates exceptional performance; it requires on average 0.05 s to resolve each query record. This is due to the bounded blocks of T , which results in constant query times. TIME, which uses the time window technique to bound the number of records, scores also constant times, which are though twice as much as those of DH, as Fig. 6(a) suggests. On the other hand, TASK employs two inverted indexes to maintain in main memory the record pairs as well as these records surpassing a predefined threshold for oversized blocks. These indexes maximized main memory usage, which eventually resulted in slow responses during the resolution of the query records. Specifically, TASK performed 300% and 100% slower than DH and TIME, respectively. To be fair, we compared the query times for SPK, which relies solely on persistent storage, and DB of DH in a separate set of experiments.

Using NCVR, whose results are shown in Fig. 6(b), we observe a slight drop of the recall and precision rates by 6 and 7 percentage units, respectively, compared with OAG. This is due to the larger number of blocks generated using $k = 6$ for DB, which are summarized in T using $k_\phi = 4$. This has the side-effect to eventually result in more evicted records from T . Increasing w would lead to higher recall rates but also to additional memory overhead due to the increased number of slots of each block. The larger number of blocks deteriorated the performance for the rest of our competitors especially in terms of their precision rates. The fast responses of DH to each query record compensate for the corresponding loss of recall, ensuring a nice balance between speed and accuracy.

DH for the largest data set, DBLP, effectively managed recall without compromising speed, as Fig. 6(c) clearly indicates. Specifically, recall is maintained slightly lower than 80% resolving the corresponding queries in nearly 0.1 s. We doubled the number of slots for each block $w = 1000$ so as to accommodate a larger number of records and reduce the number of evictions. We measured the number of blocks of DB and T , which were on average 700K ($k = 6$) and 40K ($\phi = 4$), respectively, for each hash table. These numbers suggest a summarization factor $\frac{1}{17.5}$ for T . This association between the blocks of DB and T holds only for their first 36 hash tables, matching the number of hash tables in T . We also observed that although the distribution of the number of records in each block was not uniform, each query targeted blocks, whose number of records varied greatly, which eventually accounted for the sublinear query times. Furthermore, we note that an eviction of a record from a block of a hash table, does not imply its removal from T due to the insertion of that record into totally L_ϕ hash tables. Therefore, it may be found in another block and be matched with the corresponding query record. TASK pushed the utilization of main memory, which eventually terminated throwing a fatal exception. Although SPK performed equally well with DH in terms of recall, we noticed that their F1 scores exhibit a consistent pattern: DH outperforms SPK by a margin of nearly 10 percentage units. This is justified by the superiority of DH in precision rates, which exceed SPK's by an average of 43%.

Table 3

The query times for DH, using DB, and SPK.

	OAG	NCVR	DBLP
DH	0.32 $\pm$ 0.01	0.34 $\pm$ 0.01	1.29 $\pm$ 0.05
SPK	0.36 $\pm$ 0.01	0.38 $\pm$ 0.02	2.51 $\pm$ 0.03

We, then, measured the average query times and standard deviations of DH, using DB, and SPK, which are quoted in Table 3. For OAG and NCVR, both DH and SPK demonstrated nearly identical performance with tight concentrations around their averages. However, in the case of DBLP, DH proved to be twice as fast as SPK due to its sublinear complexity during matching. On the other hand, SPK experienced delays in resolving queries due to the large volume of records associated with each distinct 3-gram during its indexing process.

6. Practical considerations and implementation insights

Throughout the development of DynaHash¹³ and conduction of its experimental evaluation, several milestones were reached and significant conclusions were drawn:

Sublinear query time for DB. In order to formulate refined blocks to achieve sublinear time complexities in the total number of blocked records for DB, parameter k , which denotes the length of the MinHash keys, should be properly set. Specifically, it should be a function of the data sets at hand by sampling a small number of records and testing the query times. Having refined blocks in DB, with proper tuning of k , allows T to maintain a more refined summarized blocking structure with less evictions.

Constant query time for T . The query time of DynaHash depends on w and not on the size of the underlying data sets. We observe in Figs. 6(a) and 6(b) that the average time to resolve a query record remains constant by setting $w = 500$. It increases for DBLP, shown in Fig. 6(c), where we had doubled w .

Balancing between recall and main memory consumption. We varied ϕ to measure the average number of blocks of DB that are summarized by a single block of T . We set ϕ equal to 3, 4, and 5, where 16, 5, and 2 blocks of DB were summarized, respectively. We have chosen $\phi = 4$, because it balanced efficiently the recall rates and number of blocks maintained in T . Setting a lower ϕ would deteriorate recall due to the larger number of evictions. On the other hand, a higher ϕ would yield an increased number of blocks maintained in T .

Balancing between recall, precision, and query time. Setting the Jaccard similarity threshold θ to 0.5 was a good trade-off between recall and query time for most of the data sets used, because θ affects the number L (and L_ϕ) of the generated hash tables. Relaxing θ would not only result in a larger number of hash tables, leading to increased storage requirements and longer query times, but it would also decrease precision rates by formulating unrelated pairs.

Key-value store. In the context of implementing DB, any embeddable key-value store can be used. The sole requirement for this key-value store is to offer an iterator capable of retrieving all objects linked to a partial key. Specifically, each key was created in this artificial namespaces:

```
<#hash table ! MinHash key : timestamp>.
```

where the timestamp was used as a workaround to simulate the insertion of multiple records into a single block. Initially, we used LevelDB¹⁴ and then with slight modifications, we migrated our implementation to using RocksDB, because it was three times faster than LevelDB.

¹³ The implementation of the proposed methods is available at the following repository <https://github.com/dimkar121/DynaHash>.

¹⁴ <https://github.com/google/leveldb>

Length of the MinHash vectors. Setting the length of the MinHash vectors to $m = 116$, according to the theoretical framework introduced in [26], maintained the similarities of both short blocking keys, like these of NCVR, and larger ones, like the blocking keys of DBLP or OAG.

Balancing between query time and space requirements in Multi-Probe matching. By increasing ω , we can achieve a significant reduction of the required number of hash tables. However, this comes at the cost of increased query times due to the utilization of the corresponding BK-Trees to indicate the neighboring blocks.

Setting threshold θ . The threshold θ should be set at a level that will allow us to adjust it to the desired value, in order to rank the results, depending on the sensitivity of the setting. Multi-probe matching could be an alternative to reduce the space requirements of a low threshold.

7. Conclusions and future work

In this paper, we introduced DynaHash, a novel and efficient randomized blocking structure for streaming record linkage. DynaHash leverages MinHash and Hamming LSH to provide theoretical guarantees on accuracy with sub-linear query times. Its two-component architecture – a persistent *DB* for completeness and a bounded in-memory summary *T* for rapid, partial results – offers a flexible solution for diverse application needs. We also introduced a Multi-Probe matching technique for Hamming LSH, demonstrating that space requirements can be reduced by up to 86% while maintaining accuracy. Moreover, DynaHash includes a ranking mechanism that orders the results returned based on their similarity to the query. Our extensive experimental evaluation on six real-world datasets shows that DynaHash consistently outperforms state-of-the-art methods, delivering superior recall and query times that are at least twice as fast as competitors.

In the future, we aim to integrate DynaHash with distributed platforms such as Apache Spark or Apache Kafka, leveraging their scalability and real-time processing capabilities to investigate the applicability of our blocking and matching algorithms in large-scale distributed settings. This integration will enable us to process massive data sets more efficiently, by harnessing parallelism and distributed computing to handle high-throughput, low-latency scenarios. Specifically, we plan to explore the possibility of receiving streaming records from multiple compute nodes operating simultaneously. Each node will maintain and utilize its own local blocking structure to optimize initial candidate generation, ensuring low computational overhead and minimal data transfer. Despite the distributed nature of the blocking process, matching will ultimately be performed in a unified and consistent manner, employing a global perspective to reconcile potential duplicates across nodes. This approach not only aims to preserve accuracy and integrity but also to ensure scalability and robustness in dynamic, real-world applications.

CRedit authorship contribution statement

Dimitrios Karapiperis: Writing – original draft, Software, Methodology, Formal analysis, Conceptualization. **Christos Tjortjis:** Writing – review & editing, Supervision, Project administration. **Vassilios S. Verykios:** Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data are included in the repository that contains the code.

References

- [1] P. Christen, Data matching - concepts and techniques for record linkage, entity resolution, and duplicate detection, in: Data-Centric Sys. and Appl, Springer, 2012.
- [2] W. Ren, X. Lian, K. Ghazinour, Online topic-aware entity resolution over incomplete data streams, in: SIGMOD, 2021, pp. 1478–1490.
- [3] D. Karapiperis, C. Tjortjis, V. Verykios, A randomized blocking structure for streaming record linkage, in: PVLDB, 2023, pp. 2783–2791.
- [4] L. Gazzari, M. Herschel, End-to-end task based parallelization for entity resolution on dynamic data, in: ICDE, 2021, pp. 1248–1259.
- [5] T. Araujo, K. Stefanidis, C. Pires, J. Nummenmaa, T.P. de Nobrega, Schema-agnostic blocking for streaming data, in: SAC, 2020, pp. 80–91.
- [6] D. Karapiperis, A. Gkoulalas-Divanis, V. Verykios, Summarization algorithms for record linkage, in: EDBT, 2018, pp. 73–84.
- [7] D. Karapiperis, A. Gkoulalas-Divanis, V. Verykios, Efficient record linkage in data streams, in: Big Data, 2020, pp. 523–532.
- [8] G. Papadakis, G. Alexiou, G. Papastefanatos, G. Koutrika, Schema-agnostic vs schema-based configurations for blocking methods on homogeneous data, in: PVLDB, 2015, pp. 315–323.
- [9] P. Christen, A survey of indexing techniques for scalable record linkage and deduplication, TKDE 12 (9) (2012) 1537–1555.
- [10] G. Papadakis, G. Papastefanatos, G. Koutrika, Supervised meta-blocking, in: PVLDB, 2014, pp. 1929–1940.
- [11] G. Papadakis, G. Koutrika, T. Palpanas, W. Nejdl, Meta-blocking: Taking entity resolution to the next level, TKDE 26 (8) (2014) 1946–1960.
- [12] A.Z. Broder, M. Charikar, A. Frieze, M. Mitzenmacher, Minwise independent permutations, in: ACM STOC, 1998, pp. 327–336.
- [13] A. Andoni, P. Indyk, Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions, CACM 51 (1) (2008) 117–122.
- [14] Q. Lv, W. Josephson, Z. Whang, M. Charikar, K. Li, Multi-probe LSH: Efficient indexing for high-dimensional similarity search, in: PVLDB, 2007, pp. 950–961.
- [15] F. Azzalini, S. Jin, M. Renzi, L. Tanca, Blocking techniques for entity linkage: A semantics-based approach, Data Sci. Eng. (2021) 20–38.
- [16] S.E. Whang, D. Marmaros, H. Garcia-Molina, Pay-as-you-go entity resolution, TKDE 25 (5) (2013) 1111–1124.
- [17] T. Papenbrock, A. Heise, F. Naumann, Progressive duplicate detection, TKDE 27 (5) (2015) 1316–1329.
- [18] D. Firmani, B. Saha, D. Srivastava, Online entity resolution using an oracle, in: PVLDB, vol. 9, 2016, pp. 384–395.
- [19] G. Simonini, G. Papadakis, T. Palpanas, S. Bergamaschi, Schema-agnostic progressive entity resolution, in: ICDE, 2018, pp. 53–64.
- [20] L. Gazzari, M. Herschel, Progressive entity resolution over incremental data, in: EDBT, 2022, pp. 80–91.
- [21] L. Gagliardelli, G. Papadakis, G. Simonini, S. Bergamaschi, T. Palpanas, GSM: A generalized approach to supervised meta-blocking for scalable entity resolution, Inf. Syst. 120 (2024).
- [22] A. Gruenheid, X. Dong, D. Srivastava, Incremental record linkage, in: PVLDB, 2014, pp. 697–708.
- [23] E. Ioannou, W. Nejdl, C. Niederee, Y. Velegrakis, On-the-fly entity-aware query processing in the presence of linkage, PVLDB 3 (1) (2010) 429–438.
- [24] H. Altwaijry, D. Kalashnikov, S. Mehrotra, Query-driven approach to entity resolution, in: PVLDB, vol. 6, 2013, pp. 1846–1857.
- [25] I. Bhattacharya, L. Getoor, L. Licamele, Query-time entity resolution, in: KDD, 2006, pp. 529–534.
- [26] D. Karapiperis, C. Tjortjis, V. Verykios, A suite of efficient randomized algorithms for streaming record linkage, TKDE 36 (7) (2024) 2803–2813.
- [27] D. Paulsen, Y. Govind, A. Doan, Sparkly: A simple yet surprisingly strong TF/IDF blocker for entity matching, in: PVLDB, 2023, pp. 1507–1519.
- [28] S. Thirumuruganathan, H. Li, N. Tang, M. Ouzzani, Y. Govind, D. Paulsen, G. Fung, A. Doan, Deep learning for blocking in entity matching: a design space exploration, in: PVLDB, 2021, pp. 2459–2472.
- [29] R. Wang, Y. Li, J. Wang, Sudowoodo: Contrastive self-supervised learning for multi-purpose data integration and preparation, in: ICDE, 2023, pp. 1502–1515.
- [30] J. Mugeni, T. Amagasa, A graph-based blocking approach for entity matching using contrastively learned embeddings, SIGAPP Appl. Comput. Rev. 22 (2023) 37–46.
- [31] J. Mugeni, T. Amagasa, A graph-based blocking approach for entity matching using pre-trained contextual embedding models, in: SAC, 2022, pp. 357–364.
- [32] T.B. Araújo, V. Efthymiou, V. Christophides, E. Pitoura, K. Stefanidis, TREATS: Fairness-aware entity resolution over streaming data, Inf. Syst. 129 (2025).
- [33] N. Shahbazi, N. Danevski, F. Nargesian, A. Asudeh, D. Srivastava, Through the fairness lens: Experimental analysis and evaluation of entity matching, PVLDB 16 (11) (2023) 3279–3292.

- [34] M. Yu, G. Li, D. Deng, J. Fe, String similarity search and join : A survey, *Front. Comput. Sci.* 10 (2016) 399–417.
- [35] M. Slaney, Y. Lifshits, J. He, Optimal parameters for locality-sensitive hashing, *Proc. IEEE* 100 (2012) 2604–2623.
- [36] B. Bloom, Space/time trade-offs in hash coding with allowable errors, *CACM* 13 (7) (1970) 422–426.
- [37] M. López, R.A. Chavoya, R. Moncada, A new data structure for efficient spatial query processing: The kb-tree, *Int. J. Geogr. Inf. Sci.* 35 (2021) 276–296.
- [38] A. Sinha, Z. Shen, Y. Song, H. Ma, D. Eide, B. Hsu, K. Wang, An overview of microsoft academic service (MAS) and applications, in: *WWW*, 2015, pp. 243–246.